

Partie 2: Programmation Python

CHAPITRE 1

Les Structures de Base de Python

Module : Informatique 1 : Algorithmique 1/Python

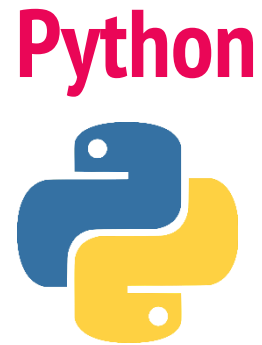


Pr. Mariya OUAISSA
E-mail : m.ouaissa@uca.ac.ma
Département : Informatique FSSM

Année Universitaire : 2023/2024

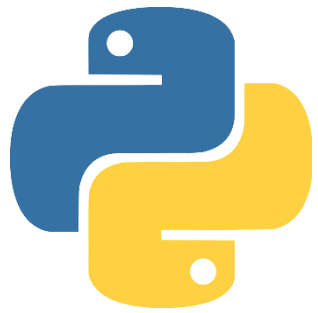
Les structures de base de python



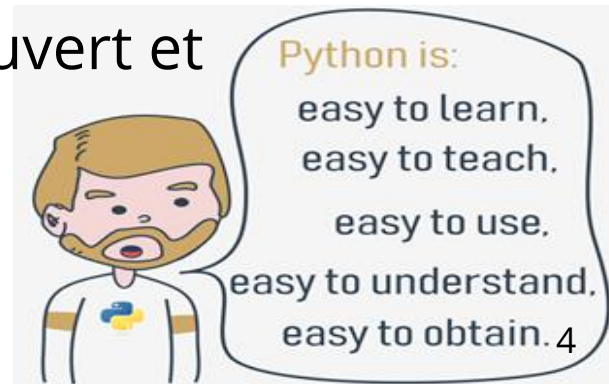


Python est **un langage de programmation Open Source**, orienté objet, de haut niveau. Il fut créé en 1991 par le développeur **Guido Van**. Il s'agit d'un langage généraliste c'est-à-dire qu'il peut être utilisé pour développer à peu près n'importe quoi, grâce à de nombreux outils et bibliothèques.

POURQUOI PYTHON ?



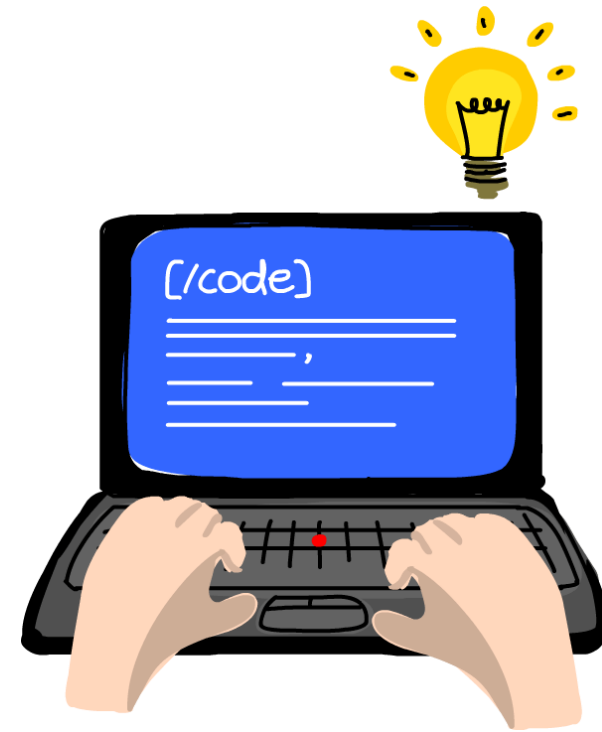
- **Facile à apprendre** - le temps nécessaire pour apprendre Python est plus court que pour de nombreux autres langages
- **Facile à utiliser** pour écrire de nouveaux logiciels - il est souvent possible d'écrire du code plus rapidement en utilisant Python
- **Facile à comprendre** - il est aussi souvent plus facile de comprendre le code de quelqu'un d'autre plus rapidement s'il est écrit en Python
- **Facile à obtenir**, installer et déployer - Python est gratuit, ouvert et multiplateforme;



La programmation et les types de programmes

Programmation

- **Langage machine** : directement compréhensible par la machine
- **Langage de programmation** : doit être compilé ou interprété pour être compris par la machine.

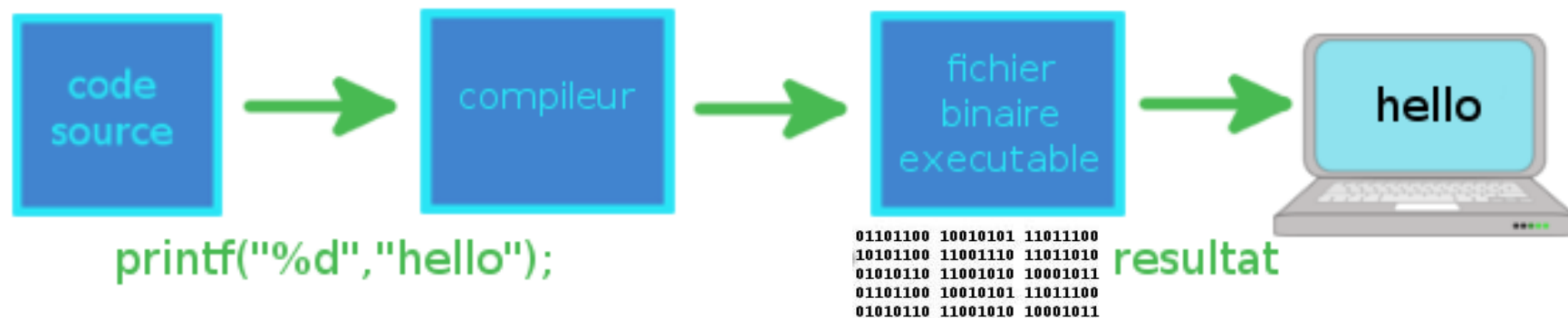


On peut distinguer deux grands types de langages :

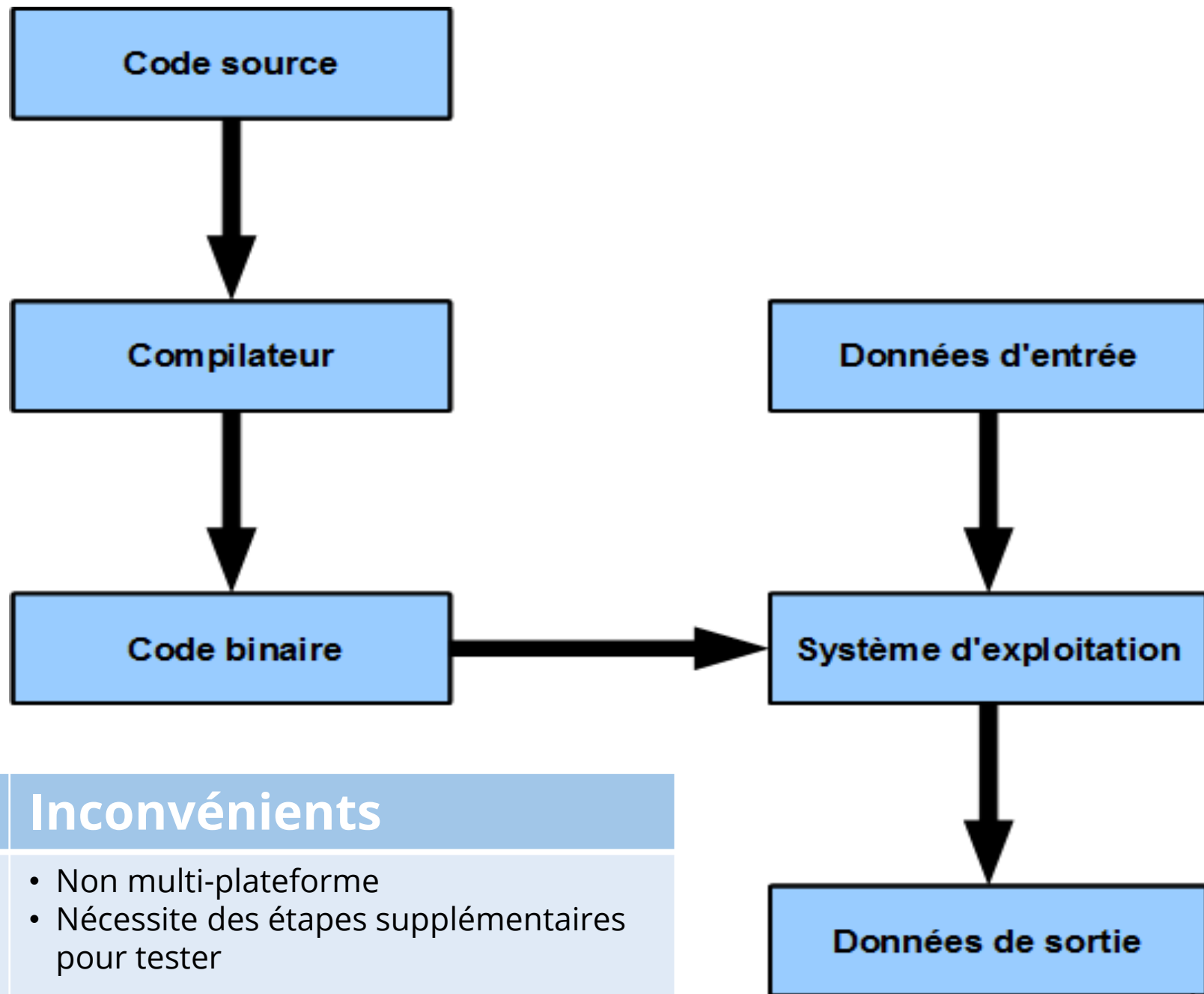
- **langages interprétés** : Java (+ JavaScool) et Python ;
- **langages compilés** : C, C++, Pascal et OCaml.

Langage compilé

- Dans ces langages, le code source (celui que vous écrivez) est tout d'abord compilé, par un logiciel qu'on appelle *compilateur*, en un *code binaire* qu'un humain ne peut pas lire mais qui est très facile à lire pour un ordinateur. C'est alors directement le système d'exploitation qui va utiliser le code binaire et les données d'entrée pour calculer les données de sortie :



Langage compilé



Avantages

- Programme immédiatement disponible à démarrer
- Plus rapide, car il est optimisé pour le CPU
- Le code source est privée

Inconvénients

- Non multi-plateforme
- Nécessite des étapes supplémentaires pour tester

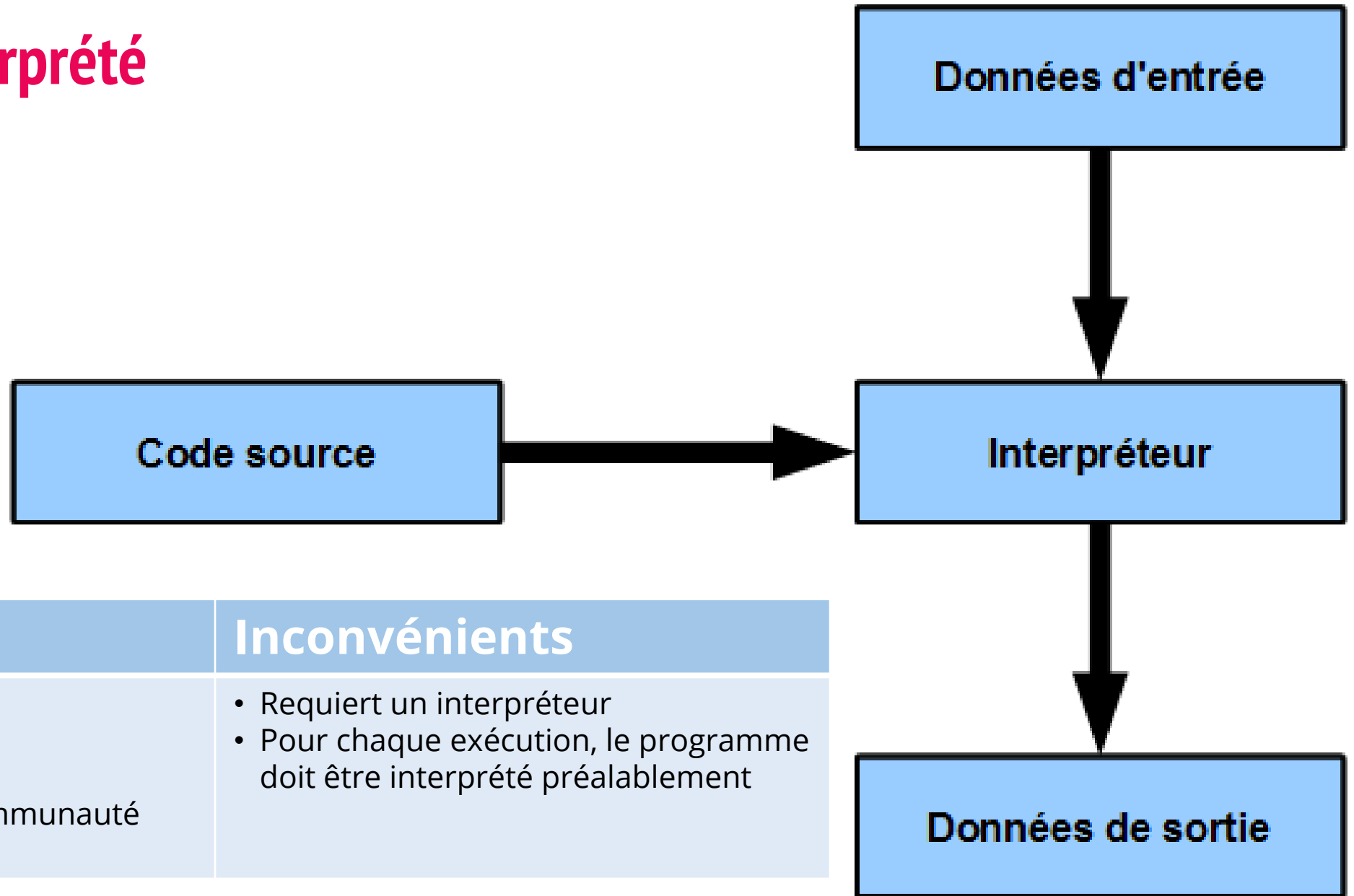
Langage interprété

- Dans ces langages, le code source (celui que vous écrivez) est interprété, par un logiciel qu'on appelle *interpréteur*. Celui-ci va utiliser le code source et les données d'entrée pour calculer les données de sortie :



- L'interprétation du code source est un processus « pas à pas » : l'interpréteur va exécuter les lignes du code une par une, en décidant à chaque étape ce qu'il va faire ensuite.

Langage interprété



Avantages	Inconvénients
• Multi-plateforme • Simple à tester • Facile à débbugger • Code source public (Communauté open source)	• Requier un interpréteur • Pour chaque exécution, le programme doit être interprété préalablement

Structures de données

Variables

Affectation de données

Types de données

Opérations

Affichage/Ecriture

Convention de nommage

Pas de caractères spéciaux (ex: @ge)

Pas d'espace

Ne pas commencer par un chiffre

Utiliser des underscores(_)

Utiliser des noms qui puissent directement commenter votre code (ex: Note_Stagiaire)

Eviter les **mots réservés** par Python

False	Class	finally	is	return
None	continue	for	lambda	try
True	def	from	nonlocal	while
and	del	global	not	with
as	elif	if	or	yield
assert	else	import	pass	
break	except	in	raise	

Les règles à suivre

pour nommer une variable

75Paris

Commence par un chiffre

Site-Web

Contient un tiret (-)

#lien video

Contient un dièse (#) et un espace

True

Utilise un mot réservé

Les conventions

pour nommer une variable

75Paris

Paris75

paris_75

Site-Web

Site_Web

site_web

#lien video

lienVideo

lien_video

True

true

true



Une Variable

- Est un nom qui sert à réserver un emplacement de la mémoire destinée à recevoir une valeur (Donnée)
- Une variable dans Python est constituée de trois éléments : **son nom**, son **type** et sa **valeur**
 - En python, on déclare les variables, il suffit de préciser son nom et sa valeur on dit que python est un langage à **type dynamique**
- Pour **modifier** la valeur d'une variable=>Il suffit simplement d'assigner une nouvelle valeur à la variable.



Python est un langage **dynamique** et fortement typé

- ▶ Pas besoin de dire à Python quel est le type de la variable
- ▶ On peut changer le type d'une variable à tout moment

Dynamique (Python)

```
ma_variable = 5  
ma_variable = "Bonjour"
```

Statique (C++)

```
int ma_variable = 5;  
string ma_variable = "Python";
```


L'INSTRUCTION D'AFFECTATION

L'instruction d'affectation permet d'attribuer une valeur (non définitive) à une variable déclarée.
L'affectation s'effectue en utilisant le symbole (=).

```
spam = 5  
ping = 9
```

- On peut affecter à une variable le contenu d'une autre variable

```
spam = ping
```

- On peut incrémenter la valeur d'une même variable sans utiliser une deuxième variable

```
spam = spam + 1
```

1. # Écrivez votre code ci-dessous!

2. `A = 5`

3. `B = 3`

4. `C = A + B`

5. `A = 3`

6. `B = C - 2`

7. `print "A =", A`

8. `print "B =", B`

9. `print "C =", C`

`A = 3`

`B = 6`

`C = 8`

L'INSTRUCTION D'AFFECTATION

L'instruction d'affectation permet d'attribuer une valeur (non définitive) à une variable déclarée.
L'affectation s'effectue en utilisant le symbole (=).

```
spam = 5  
ping = 9
```

- On peut affecter à une variable le contenu d'une autre variable

```
spam = ping
```

- On peut incrémenter la valeur d'une même variable sans utiliser une deuxième variable

```
spam = spam + 1
```

```
1. # Écrivez votre code ci-dessous!  
2. A = 5  
3. B = A + 4  
4. A = A + 1  
5. B = A - 4  
6. C = A - B  
7. Print " A = ", A  
8. print " B = ", B  
9. print " C = ", C
```

```
A = 6
```

```
B = 2
```

```
C = 4
```

Les types de données en Python

- Un type de données est simplement une façon de classer différents types de variables. Quand vous avez un fruit, vous pouvez le classer dans un type spécifique comme pomme ou orange par exemple. Quand vous avez une variable, l'ordinateur doit la classer dans un type spécifique aussi : un **type de données**.

Les types de données les plus simples, ou *primitifs*, utilisés dans Python sont :

- **int** : Les entiers (*Integers* en anglais)
- **float** : Les virgules flottantes (*Float* en anglais)
- **str** : Les chaînes de caractères (*Strings* en anglais)
- **bool** : Les booléens



Les types de variables

Donner un type à une variable consiste à définir l'ensemble des valeurs que peut prendre cette variable ainsi que les opérations qui peuvent lui être appliquées

- **Types numériques:**

- **Booléen** : True ou False
- **Entier** : -32 768 à 32 767
- **Réel** : $-1,79 \times 10^{38}$ à $1,79 \times 10^{38}$

- **Types caractères**

- Un caractère peut être une lettre (**a** à **z** ou **A** à **Z**) ou bien un symbole (**+, &, \$, #**) ou encore un chiffre (**0** à **9**)

- **Types chaînes de caractères:**

- Une chaîne de caractères est une suite finie de caractère (**a** à **z** ou **A** à **Z**) ou bien des symboles (**?, !, @, _**) ou encore des chiffres de **0** à **9**

Types de données en Python

- Pour afficher le type de la variable : **type(nom_variable)**
- Pour détruire une variable : **del nom_variable**

```
>>> a=1
>>> b="Bonjour"
>>> c=2.5
>>> d='m@test'
>>> type(d)
<class 'str'>
>>> type(a)
<class 'int'>
>>> type(b)
<class 'str'>
>>> type(c)
<class 'float'>
>>> |
```

```
>>> del d
>>> d
Traceback (most recent call last):
  File "<pyshell#11>", line 1, in <module>
    d
NameError: name 'd' is not defined
>>> a
1
>>> b ; c
'Bonjour'
2.5
>>> |
```

Les opérations sur les types numériques

- Tout comme les nombres dans Excel ou avec une calculatrice, vous pouvez réaliser toutes sortes d'opérations arithmétiques avec des entiers et des virgules flottantes dans Python:
 - **$x + y$: la somme de x et y (x plus y).**
 - **$x - y$: la différence entre x et y (x moins y).**
 - **$x * y$: le produit de x et y (x fois y).**
 - **x / y : le quotient de x et y (x divisé par y).**
 - **$x \% y$: le reste de x divisé par y (le reste de la division entière).**
 - **$x ** y$: x à la puissance y**
 - **$x // y$: la division entière de x sur y**
- Les symboles $+$, $-$, $*$, $/$, $**$, $//$ et $\%$ sont appelés opérateurs, car ils réalisent des opérations sur les variables.

Opérateurs de raccourcis

- Très souvent, nous voulons utiliser une seule et même variable à la fois à droite et à gauche de l'opérateur = .
- Par exemple, si nous devons calculer une série de valeurs successives de puissances de 2 :
x=x2**
- Python vous offre une manière raccourcie d'écrire des opérations comme celle-ci, qui peuvent être codées comme suit: **x**=2**
- Si op est un opérateur à deux arguments (c'est une condition très importante) et l'opérateur est utilisé dans le contexte suivant:
variable = variable op expression
- Il peut être simplifié et illustré comme suit:
 - **variable op= expression**
- L'opérateur **+=** effectue une addition puis affecte le résultat à la même variable. Cette opération s'appelle une « incrémentation ».
- Les opérateurs **-=**, ***=** et **/=** se comportent de manière similaire pour la soustraction, la multiplication et la division.

Opérateur de raccourcis

- Jetez un œil aux exemples ci-dessous.
Assurez-vous de tous les comprendre.

```
i = i + 2 * j
```



```
i += 2 * j
```

```
var = var / 2
```



```
var /= 2
```

```
rem = rem % 10
```



```
rem %= 10
```

```
j = j - (i + var + rem)
```



```
j -= (i + var + rem)
```

```
x = x ** 2
```



```
x **= 2
```


Opérations sur les chaînes de caractères

- Pour les chaînes de caractères, deux opérations sont possibles, l'addition et la multiplication :

```
1 >>> chaine = "Salut"  
2 >>> chaine  
3 'Salut '  
4 >>> chaine + " Python"  
5 'Salut Python '  
6 >>> chaine * 3  
7 'SalutSalutSalut '
```

- L'opérateur d'addition + concatène (assemble) deux chaînes de caractères.
- L'opérateur de multiplication * entre un nombre entier et une chaîne de caractères duplique (répète) plusieurs fois une chaîne de caractères.

Attention

Vous observez que les opérateurs + et * se comportent différemment s'il s'agit d'entiers ou de chaînes de caractères : 2 + 2 est une addition alors que "2" + "2" est une concaténation. On appelle ce comportement **redéfinition des opérateurs**.

Opérateurs de chaîne – Réplication/duplication

- Le signe * (astérisque), lorsqu'il est appliqué à une chaîne et un nombre (ou un nombre et une chaîne, car il reste commutatif dans cette position) devient un opérateur de réplication :

```
string * number  
number * string
```

- Il réplique la chaîne le même nombre de fois spécifié par le nombre. Par exemple:

✓ `"James" * 3` donne `"JamesJamesJames"`

✓ `3 * "an"` donne `"ananan"`

✓ `5 * "2"` Ou `"2" * 5` donne `"22222"`

- Remarque** : Un nombre inférieur ou égal à zéro produit une chaîne vide

Les conversions



Les fonctions `int()`, `float()` et `str()` permettent de convertir un nombre quelconque ou une chaîne de caractères respectivement en un entier (arrondi), en un nombre réel et en chaîne de caractère.

```
x = int(3.5)
y = float(3)
z = int("3")
print("x:", type(x), "    y:", type(y), "    z:", type(z))

# x: <class 'int'>    y: <class 'float'>    z: <class 'int'>
```

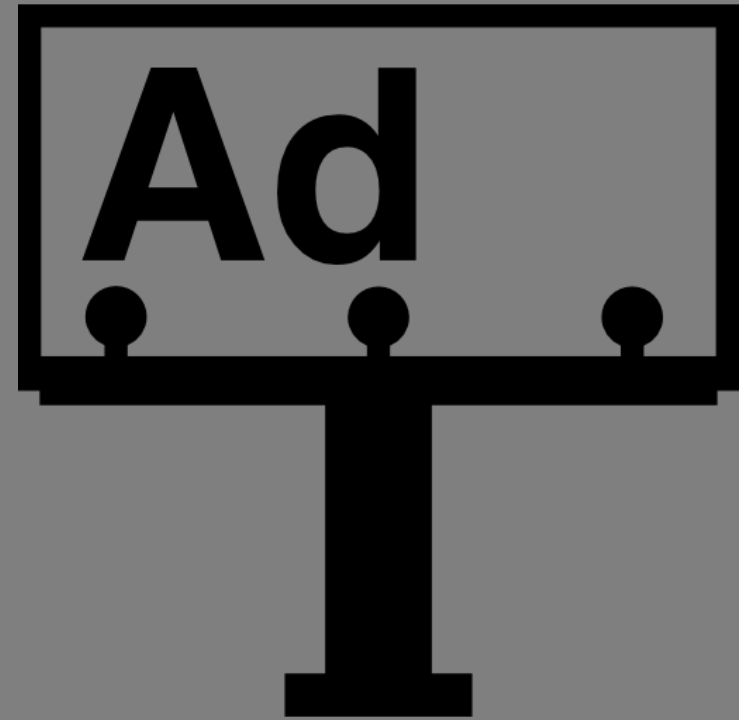
Il peut arriver que la conversion en un nombre entier ne soit pas directe. Dans l'exemple qui suit, on cherche à convertir une chaîne de caractères en entier mais cette chaîne représente un réel. Il faut d'abord la convertir en réel puis en entier, c'est à ce moment que l'arrondi sera effectué.

```
i = int("3.5") # provoque une erreur
i = int(float("3.5")) # fonctionne
```

Opérations sur les chaînes de caractères - Application

```
x = int(1)    # x will be 1
y = int(2.8)  # y will be 2
z = int("3")  # z will be 3
x = float(1)   # x will be 1.0
y = float(2.8) # y will be 2.8
z = float("3") # z will be 3.0
w = float("4.2") # w will be 4.2
x = str("s1")  # x will be 's1'
y = str(2)     # y will be '2'
z = str(3.0)   # z will be '3.0'
```

L'affichage



La fonction `print()`

- la fonction `print()` affiche l'argument qu'on lui passe entre parenthèses **et** un retour à ligne.
- Ce retour à ligne supplémentaire est ajouté par défaut. Si toutefois, on ne veut pas afficher ce retour à la ligne, on peut utiliser l'argument par « mot-clé » `end` :

```
>>> print(" helloFSSM ",end="!")
hello FSSM!
>>> print ("helloFSSM !")
hello FSSM!
```

- Une autre manière de s'en rendre compte est d'utiliser deux fonctions `print()` à la suite. Dans la portion de code suivante, le caractère « `;` » sert à séparer plusieurs instructions Python sur une même ligne :

```
>>> print("salam");print('FSSM')
salam
FSSM
```

La fonction `print()`

- La fonction **`print()`** peut également afficher le contenu d'une variable quel que soit son type. Par exemple, pour un entier :

```
>>> var=3
>>> print(var)
3
```

- Il est également possible d'afficher le contenu de plusieurs variables (quel que soit leur type) en les séparant par des virgules :

```
>>> a=32
>>> prenom= 'Mohammed'
>>> print(prenom, "a", a, 'ans')
```

Mohammed a 32 ans

Pour afficher plusieurs éléments de texte sur une seule ligne, nous avons utilisé le séparateur « , » entre les différents éléments,

La fonction print()

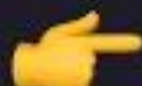
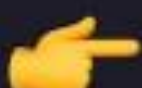
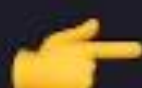
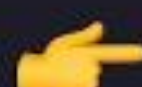
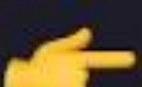
- Python a également ajouté un espace à chaque fois que l'on utilisait le séparateur « , ». On peut modifier ce comportement en passant à la fonction print() l'argument par mot-clé sep :

```
>>> print(prenom, "a", a, "ans", sep="-")
```

Mohammed-a-32-ans

- Pour afficher deux chaînes de caractères l'une à côté de l'autre, sans espace, on peut soit les concaténer, soit utiliser l'argument par mot-clé sep avec une chaîne de caractères vide :

```
>>> animal1="chat"
>>> animal2="souris"
>>> print(animal1,animal2)
chat souris
>>> print(animal1+animal2)
chatsouris
>>> print(animal1,animal2,sep=" ")
chat souris
```


\a		caractère d'appel (BEL)
\b		caractère de retour arrière
\f		saut de page
\n		retour à la ligne
\r		retour chariot
\t		tabulation horizontal
\v		tabulation verticale

Lecture

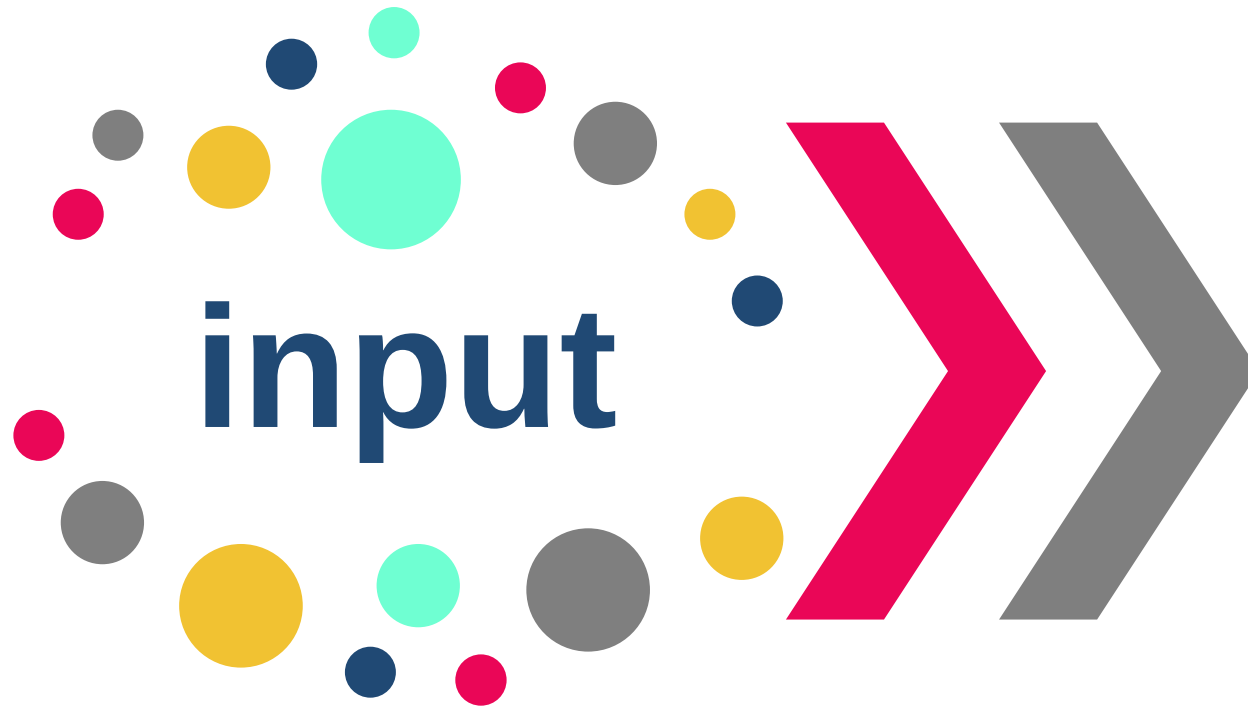


La fonction `input()`

- La fonction **`input()`** est capable de lire les données saisies par l'utilisateur et de renvoyer les mêmes données au programme en cours d'exécution.

```
>>> nombre = input("Entrez un nombre: ")
Entrez un nombre: 5
>>> print(nombre)
5
```

- Le programme peut manipuler les données, rendant le code vraiment interactif.
- Pratiquement tous les programmes lisent et traitent les données. Un programme qui n'obtient pas l'entrée d'un utilisateur est un programme sourd .



Chaîne de
caractère

La fonction input()

- Jetez un œil a notre exemple:

```
print("écrivez quelque chose : ")
anything=input()
print("vous avez écrit : ",anything,"Bravo")

===== RESTART: C:\Users\toshiba\Documents\test2.py =====
écrivez quelque chose :
bonjour
vous avez écrit :  bonjour Bravo
>>>
```

- Le programme précédant invite l'utilisateur a saisi certaines données depuis la console (très probablement a l'aide d'un clavier).
- La fonction input() est invoquée sans arguments (c'est la manière la plus simple d'utiliser la fonction); la fonction bascule la console en mode d'entrée ; vous verrez un curseur clignotant, et vous serez en mesure de saisir quelques frappes, en appuyant sur la touche Entrée ;
- Toutes les données saisies seront envoyées a votre programme via le résultat de la fonction;

La fonction input()

Remarque

- vous devez affecter le résultat à une variable, Ceci est crucial - manquer cette étape entraînera la perte des données saisies;
- Puis nous utilisons la fonction print() pour afficher les données obtenues

La fonction input()

- La fonction input() peut faire autre chose: elle peut inviter l'utilisateur a saisir son entrée sans aucune aide de print().
- Nous avons un peu modifié notre exemple, regardez le code:

```
test2.py - C:\Users\toshiba\Documents\test2.py (3.6.8)
File Edit Format Run Options Window Help
#print("écrivez quelque chose : ")
anything=input("écrivez quelque chose : ")
print("vous avez écrit : ",anything,"Bravo")

===== RESTART: C:\Users\toshiba\Documents
écrivez quelque chose : bonjour
vous avez écrit : bonjour Bravo
>>>
```

- la fonction input() est appelée avec un argument - c'est une chaîne contenant un message;
- le message sera affiché sur la console avant que l'utilisateur ait la possibilité d'entrer quoi que ce soit; input() va ensuite lire le texte saisi et l'affecter à la variable anything.
- Cette variante de l'invocation de input() simplifie le code et le rend plus clair.

En résumé

- ▶ Python est un langage dynamique et fortement typé.
 - ▶ On n'a pas besoin d'indiquer le type d'une variable lors de sa définition.
 - ▶ On peut changer le type d'une variable n'importe quand.
- ▶ Pour réaliser certaines opérations, on doit convertir les variables d'un type à un autre.

Conversion des types de variables

```
>>> a = "5"  
>>> a = int(a)  
>>> print(a)  
5
```

```
>>> a = 5  
>>> b = "10"  
>>> b = int(b)  
>>> print(a + b)  
15
```

Avec Python 3

```
>>> nombre = input("Entrez un nombre : ")
```

```
Entrez un nombre : 10
```

```
>>> print(nombre)
```

```
10
```

```
>>> print(type(nombre))
```

```
<class 'str'>
```

La fonction `input()`

1

Cette fonction provoque une interruption dans le programme courant.

2

L'utilisateur est invité à entrer des caractères au clavier et à terminer avec <Enter>.

3

Lorsque cette touche est enfoncée, l'exécution du programme se poursuit, et la fonction fournit en retour une valeur correspondant à ce que l'utilisateur a entré.

4

Cette valeur peut alors être assignée à une variable quelconque.

On peut invoquer la fonction `input()` en laissant les parenthèses vides. On peut aussi y placer en argument un message explicatif destiné à l'utilisateur. Exemple :

```
print ("veuillez entrer un nombre positif quelconque: ")
nn=int(input())
nn2=nn**2
print ("le carré est de %d est %d"%(nn,nn2))
```

Python 3.6.8 Shell

File Edit Shell Debug Options Window Help

>>>

===== RESTART: C:/Users/toshiba/Documents/test2.py

veuillez entrer un nombre positif quelconque:

2

le carré est de 2 est 4

La fonction input() – type casting

- Python propose deux fonctions simples pour spécifier un type de données et résoudre ce problème : int() et float().
- Leurs noms sont auto-commentés:
 - La fonction int() prend un argument (par exemple, une chaîne:) int(string) et essaie de le convertir en un entier;
 - La fonction float() prend un argument (par exemple, une chaîne:) float(string) et essaie de le convertir en float

```
anything = float(input("Enter a number: "))  
something = anything ** 2.0  
print(anything, "to the power of 2 is", something)
```

```
===== RESTART: C:\Users\toshiba\Documents\test2.py  
Enter a number: 5  
5.0 to the power of 2 is 25.0  
>>>
```

La fonction `input()` – type casting

- Les fonctions `print()`-`input()`- `int()`- `float()` et `srt()` ouvre beaucoup de nouvelles possibilités.
- Vous pourrez éventuellement écrire des programmes complets, accepter des données sous forme de nombres, les traiter et afficher les résultats.

Quelques fonctions importantes

round(5.76543, 2)

- Pour préciser le nombre de digits à afficher après la virgule

Type(variable)

- Pour afficher le type de donnée de cette variable

a = "Hello, World!"

print(len(a))

- Afficher le nombre de lettre de cette chaîne

txt = "The best things in life are free!"

print("free" in txt)

- On cherche le mot free dans la chaîne txt et l'instruction suivante fait l'inverse

print("expensive" not in txt)

Manipuler les chaînes de caractères

Commençons par la plus simple `len (...)`, qui obtient la longueur (le nombre de caractères) d'une chaîne!

1. `# La sortie sera le nombre de lettres en "Norwegian Blue"!`
2. `parrot = "Norwegian Blue"`
3. `print len (parrot)`

Changer la casse : upper / lower /

"Bonjour".**upper()** #majuscule

"Bonjour".**lower()** #minuscule

Utilisée pour unifier la recherche d'un utilisateur et sans prendre en compte si les lettres sont en maj ou min

"bonjour tous le monde !".**capitalize()** #mettre une majuscule au début de la phrase

"bonjour tout le monde !".**title()** #mettre une majuscule au début de chaque mot

Utilisées si vous voulez s'assurer que chaque phrase contient une majuscule au début d'un livre ou les majuscules des mots qui composent les titres

Remplacer des éléments dans les chaînes de caractères

`"Bonjour".replace("jour","soir")` #remplacer le 1r argument par le 2eme

`"Bonjour bonjour".replace("jour","soir")` # ça marche sur toutes les occurrences de jour

`"Bonjour bonjour".replace(" ", "")` # ça marche pour les espace

`"Bonjour bonjour".replace("jour","soir").replace(" ", "")`

on peut faire plusieurs remplacement dans la même chaîne

COMMENTAIRES

COMMENTAIRES

Les commentaires rendent votre programme plus facile à comprendre. Lorsque vous regardez votre code ou que d'autres veulent collaborer avec vous, ils peuvent lire vos commentaires et facilement comprendre ce que fait votre code.

- Le signe # est pour les commentaires. Un commentaire est une ligne de texte que Python n'essayera pas d'exécuter en tant que code. C'est juste pour les humains à lire.
- Vous pouvez écrire un commentaire multi-ligne, en commençant chaque ligne avec #, cela peut être une douleur.
- Au lieu de cela, pour les commentaires sur plusieurs lignes, vous pouvez inclure le bloc entier dans un ensemble de guillemets (" " ")

```
# Écrivez votre commentaire ici
```

```
# Écrivez un autre
```

```
" " " Siégeant de ta tasse jusqu'à ce que ça  
coule,
```

```
Saint Graal.
```

```
" " "
```

Exécute